OPENCV AI COMPETITION 2026 · TECHNICAL REPORT

AutoScope

Malaria microscopy with an agent. An OpenCV 5 pipeline counts parasites and white cells on each thick-film field. An agent decides which field to read next and when to stop.

0.790 parasite F1 on 50 held-out patients	**500/µL** limit of detection at 95% sensitivity
0.98 sensitivity per patient, none called negative	**547** slides per hour on one c8g.8xlarge (Graviton4)

Live demo: d1k2o3z37c1maf.cloudfront.net
Every figure in this report is read from a committed result file by `docs/report/report_numbers.py`.

Uriel Nguefack Yefou · **Alex Bruno Kenfack Anafack**
Team AutoScope · built 26 August to 26 October 2026 · this report 24 October 2026

AutoScope is a screening aid, not a diagnostic device. It has not been clinically validated. All images are public, retrospective, de-identified research material.

1 The problem

Microscopy gives a number. A rapid test gives yes or no.

The WHO *World malaria report 2025* counts 282 million cases and 610 000 deaths in 2024. 94% of cases and 95% of deaths were in the African Region. Three quarters of those deaths were children under five.

Rapid diagnostic tests carry most of the field load. Two things limit them. *pfhrp2* gene deletions, found in 42 countries, remove the commonest rapid-test target from the parasite. Artemisinin partial resistance is detected by counting parasites on day 0, day 2 and day 3. A rapid test cannot count.

The clinical quantity is parasites per microlitre. A technician obtains it at an eyepiece by counting parasites against 200 white blood cells on a thick film. It is slow. The precision depends on how many cells were counted before the technician stopped. Where there are few microscopists, the count is skipped.

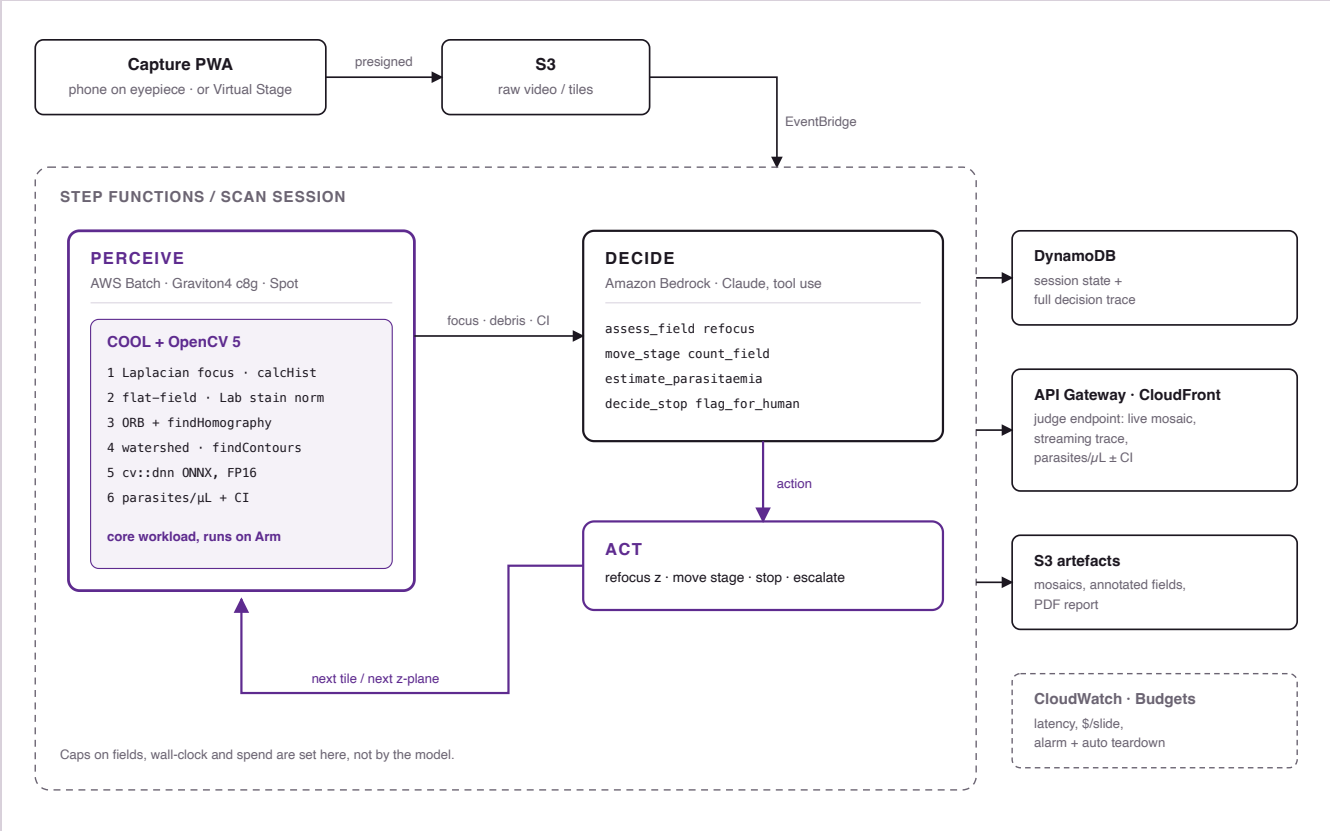
Who it is for

A district hospital or health centre that owns a microscope and employs a technician. There is no slide scanner. A phone camera is held to the eyepiece. The fields are uploaded. The count comes back with a confidence interval and the crops it was made from. A person signs the result.

2 What AutoScope does

Six OpenCV stages produce the counts. An agent decides how many fields to read. A

fixed function turns the counts into a call, and the agent has no part in that step.



All six stages are OpenCV 5, and they are listed in Section 3. The agent sits outside them: its whole input is OpenCV output and the only thing it returns is which field to read next.

The loop

A field arrives. OpenCV finds the lit disc and scores it for focus, evenness and debris. The stain is then normalised and the white cells segmented. A top-hat proposes the parasite candidates, and a small convolutional network scores each one through `cv.dnn`. The running totals go to the agent with the current density estimate and its Poisson interval. The agent picks one of five moves. When it stops, `density.decide()` reads the totals and returns the call. The trace of every move, with the agent's reason for each, is what the technician reviews.

The five moves. `src/autoscope/agent/moves.py`.

MOVE	WHEN
`count_then_move`	The field is readable. Count it and advance.
`refocus`	The focus score is low. Step the focus and look again.
`skip`	Debris, a bubble or an edge. Do not count it.
`stop`	The interval is tight enough, or the protocol's cell count is reached.
`flag_for_human`	The evidence will not settle. Escalate.

3 The OpenCV 5 pipeline

Six stages, all `imgproc` and `dnn`, pinned to `opencv-python 5.0.0.93`.

Stage 1. Field of view

A phone through an eyepiece gives a bright disc on a black surround. `fov.detect` thresholds the image, closes it with an elliptical kernel and keeps the largest component. Every later stage works inside the disc.

Stage 2. Field quality

Three numbers per field: focus, from the variance of a Laplacian inside the disc; evenness, from the illumination field; and a debris fraction. These are the inputs to `refocus` and `skip`.

Stage 3. Stain normalisation

Giemsa staining varies between laboratories more than parasites vary between patients. The pipeline works in Lab colour space and uses the `a–b` channel as the stain map. This separates nuclear purple from background at any stain intensity.

Stage 4. White cells

White cells are the denominator of every density. Nuclei are segmented from the stain map and separated with a distance transform and connected components.

Stage 5. Parasite candidates and the patch network

A morphological top-hat on the smoothed stain map leaves small dark purple objects. Local maxima above a threshold, outside the white-cell mask, are the candidates. Candidate recall against all annotated parasites is **0.906**. Each candidate is cut to a 48 px patch and scored by a small CNN, exported to ONNX and run through `cv.dnn`. The same binary runs on the laptop and on Graviton. The operating threshold is **0.53**, chosen on 20 calibration patients by the rule *closest to zero count bias*.

Stage 6. Density and interval

`density()` converts parasites per white cell to parasites per microlitre at the WHO convention of 8 000 leucocytes/µL. The confidence interval is an exact Poisson interval. At a few parasites in 200 cells, the normal approximation is too narrow.

4 The agent

The agent chooses where to look and when to stop. It does not choose the diagnosis.

The agent returns a structured move. When the move is `stop`, it also returns its own opinion of the call. That opinion is recorded and not used. The call comes from `density.decide()`, the same function that serves the rule-based policies. So the comparison between an agent and a

rule is a comparison of stopping points only.

A trace

One slide, printed from the recorded calls by `eval/render_trace.py TF124_S540`. The reasons are the model's own words, unedited.

```
slide: TF124_S540
  1  (0,0) z=0  count_field()  focus=3  even=0.95  debris=0.63  parasites=58  wbc=17
     counted, running total 58 in 17 white cells
  2  (1,0) z=0  count_field()  focus=3  even=0.97  debris=0.68  parasites=42  wbc=24
     counted, running total 100 in 41 white cells
  3  (2,0) z=0  count_field()  focus=3  even=0.97  debris=0.68  parasites=34  wbc=28
     counted, running total 134 in 69 white cells
  4  (3,0) z=0  count_field()  focus=2  even=0.95  debris=0.67  parasites=12  wbc=25
     counted, running total 146 in 94 white cells
  5  (4,0) z=0  count_field()  focus=2  even=0.95  debris=0.62  parasites=0  wbc=24
     counted, running total 146 in 118 white cells
  6  (5,0) z=0  count_field()  focus=3  even=0.96  debris=0.68  parasites=2  wbc=23
     counted, running total 148 in 141 white cells
  7  (6,0) z=0  stop           focus=3  even=0.96  debris=0.74
     settled: After 6 fields, 141 white cells, the estimate is 8,397/µL with a CI of 7,099–9,864/µL — the
  entire interval sits within the 1,000–10,000 band, the relative width is 33%, and we are near the 200 WBC
  WHO target; the density band call is unambiguous.
  read 6 fields, 148 parasites in 141 white cells
```

The smear thins as it goes: 58 parasites on the first field, then 42, 34, 12, 0 and 2, while the white cells keep coming at about 23 a field. The agent stops at six of the ten fields this slide has, with 141 white cells against the WHO protocol's 200, because the interval no longer touches a band edge. Every move of every slide is in `eval/results/agent_eval.json`, and five slides read live on AWS are on the demo page with the field images beside the reasons.

Replay without cost

Every model call is recorded to a JSONL cache, keyed on the model, the effort setting, the system prompt, the tool definitions and the observation. **15,396 calls across 15 models** ship with the repository. A changed prompt changes the key. A cache miss during replay raises instead of calling the model. The comparison re-runs with no AWS account.

5 What is already known, and what is not

Automated malaria microscopy is a solved research problem at the field level. What is not solved is the slide.

The detector. Yang et al. is the paper that goes with this dataset: a smartphone-based detector on the same 150 patients, reporting 92.6% sensitivity and 94.3% precision at the candidate level.[1] Everything measured here is read against that. Our detector is deliberately a small CNN on top of classical morphology, because it has to run through `cv.dnn` on an Arm instance beside five other OpenCV stages. The contribution lies elsewhere.

The protocol. The WHO already answers how many fields to read: count parasites against 200 white cells, or read 100 fields before calling a slide negative.[2] That protocol is a fixed budget, and it spends the same effort on a crowded slide as on an empty one. In the literature we found, how much of a slide is enough was never treated as a decision to be made per slide and measured.

What is new here. Three things. A measurement stands behind each one.

1. **The stopping decision is separated from the diagnosis and measured on its own.** Every policy in Section 7 reads the same fields with the same counter and the same `decide()`; only the stopping point changes. That is what makes an agent comparable to a rule at all, and it is why the honest answer about a fixed eight fields could be found and settled.
2. **Fifteen models through one loop, replayable for nothing.** 15,396 recorded calls ship with the repository. The finding is that model size does not predict anything and willingness to stop does: 3 of the 15 never chose `stop` once. That is not visible on any model card and costs one cheap run to check.
3. **A defect in a shipped accelerated OpenCV, diagnosed to the call and worked around.** Section 9. The function benchmark says 1.59 times faster; the pipeline says 3.8 times slower; the thread sweep says why. Function-level benchmarks are how hardware gets chosen, and here is a case where they point the wrong way.

1. Yang F, Poostchi M, Yu H, Zhou Z, Silamut K, Yu J, Maude RJ, Jaeger S, Antani S. *Deep learning for smartphone-based malaria parasite detection in thick blood smears.* IEEE Journal of Biomedical and Health Informatics, 24(5):1427–1438, 2020. `doi:10.1109/JBHI.2019.2939121`
2. WHO, *Basic malaria microscopy*, and the counting convention of 8 000 leucocytes per microlitre used throughout this report. Burden figures are from the *World malaria report 2025*, in `docs/reference/`.
3. Tanzania set: Harvard Dataverse `doi:10.7910/DVN/O2WVWA`, CC0, with its Data in Brief paper `doi:10.1016/j.dib.2024.111169`.

6 On AWS

61 Terraform resources in `us-east-1`. Five slides have been read end to end.

`infra/terraform/`, one `apply`.

SERVICE	ROLE
S3, CloudFront	Field uploads, results, the public demo page.
API Gateway, Lambda	Starts a session. Returns the trace.
Step Functions	The reading loop: assess, count, ask the agent, repeat or stop.
Batch on Graviton Spot	Runs the OpenCV container, `c8g`, arm64.
Bedrock	The agent. 15 models compared in Section 7.
DynamoDB	One row per field read, with the observation, the counts and the move. The totals are rebuilt from those rows, so they cannot drift from the audit trail.

The container is the same arm64 image the benchmark times. CI runs the test suite on x86 and arm64. Benchmark instances run on Spot and terminate themselves. The demo page is public and needs no credentials.

Every field says which binary read it. Each trace row carries the OpenCV version, the custom HAL line, the parallel framework and the machine, read from `cv.getBuildInformation()` in the job itself. A speed claim about an accelerated build is only checkable if the build travels with the data, so it appears on the demo page under the counts it produced as well as in the benchmark files.

What is logged, what is locked down, where a person decides

Observability. 3 CloudWatch log groups, 30-day retention: the API, the Step Functions state machine and the Batch jobs. The state machine logs at `ALL` with execution data on, so the log holds every field read, every agent move and the reason given, down to the final answer. Each slide also writes one DynamoDB row per move, which is what the demo page reads back. An alarm fires on more than 50 sessions started in five minutes, and two budgets, $100 a month overall and $25 for Bedrock, email on 80% forecast and 100% actual.

Security. 9 IAM roles, one per component, each scoped to the bucket, table, state machine or function it needs by ARN. All 3 S3 buckets block public access and are encrypted at rest; the demo page is served through CloudFront, not from a public bucket. The API has no write path to anything but the uploads prefix and is throttled to 10 requests a second with a burst of 20, so a loose link cannot run up a bill. Deployments use GitHub OIDC with no long-lived keys. No patient-identifying data exists: the slides are public de-identified research images.

Human control. The agent has no way to issue a diagnosis. Its moves are count, refocus, skip, stop and `flag_for_human`, and the call always comes from `decide()` on the counts. When the agent stops it may state its own opinion of the call; that opinion is recorded and never used,

and the two agreed on all 36 of the slides where it gave one. A slide it cannot settle goes to a person: 14 of 50 slides with the shipped counter. Every result carries the field images and the per-move reasons that produced it, so a reviewer can check the work.

The number alone is never the output.

7 Evaluation

The plan of 2 September set seven numeric targets. 5 of 7 were met.

Targets from `docs/plan-2026-09-02.md` against measured values.

TARGET	MEASURED	
Object-level parasite F1 ≥ 0.85, held-out NLM patients	**0.790** on 50 patients, 100 images	MISSED
Per-patient sensitivity ≥ 0.95 on infected slides	**0.98** (49/50 called positive, 1 to a person, none called negative)	MET
Per-patient specificity ≥ 0.90 on the uninfected set	**0.88** (0 of 25 clean slides called positive, 3 escalated)	MISSED
Density within ±25% of the expert count on ≥ 70% of slides	**70.0%** of 50 slides; mean absolute error 19.3%	MET
The agent's saving in fields, with the decision it settled	**393** fields against 450 for the rule, 39 bands settled	MET
Tanzania image-level AUC and the *P. vivax* result, each run once	AUC **0.850** on 2,190 images; vivax F1 **0.292**	MET
COOL against stock against x86, per function and whole-pipeline, with the build line that proves which library ran	Four arms, nine result files, every build line recorded	MET

The detector

50 held-out NLM patients, 100 images, 4,999 annotated parasites, threshold 0.53. Precision **0.805**, recall **0.776**, **F1 0.790**. Count bias -3.62%. The parasite-per-white-cell ratio tracks the expert's with Spearman correlation 0.943 and bias -1.45%.

F1 0.790 is below the 0.85 target. The slide-level call and the density are carried by the decision rule, which reads sensitivity **0.98** with none of the 50 infected slides called negative.

Specificity

25 uninfected patients, 572 fields, the frozen test half. 94.8% of fields return no detection. Pooled false parasites: 0.47 per 200 white cells. Worst field: 3.

0 of 25 clean slides called positive. 3 sent to a person. An escalation counts against specificity, which is therefore **0.88**, below the 0.90 target.

This half has been scored twice, and both numbers are here. The first scoring, on 19 September, read specificity 0.60 with 10 slides sent to a person. Seven of those ten had reached 500 white cells with the estimate under the noise floor on their final field: the reading loop did

not re-consult the decision rule on that field, so the slide was handed over when the rule would have called it negative. The loop was corrected in commit `754deee`, which made the first number a measurement of code that no longer exists, and the half was scored again on 16 October at 0.88 with 3 escalations. Sensitivity, fields read and the floor are unchanged. Both scorings are recorded in `eval/results/negative_eval.json` under `scorings`. No threshold, floor or model was chosen on this half between the two runs.

Density and limit of detection

On 50 slides, density is within ±25% of the expert count on **70%** and within ±50% on 90%. Mean absolute error 19.3%. By dilution over 40 repeats, the limit of detection at 95% sensitivity is **500 parasites/µL**. The noise floor on clean slides is 148/µL. The rule does not call a slide positive below it.

The agent against the rules

50 patients, 609 fields. Every policy reads the same fields with the same counter and the same `decide()`. Three fixed policies and 15 models on Bedrock ran through the same loop, with the same prompt and the same moves. The last column counts how many times each model chose `stop` in its recorded calls.

Shipped counter. Fields read is the cost. Models in order of fields read.

STOPPING POLICY	FIELDS READ	DENSITY ERROR	BANDS SETTLED	TO A PERSON	CHOSE STOP
WHO 200-white-cell protocol	524	26.5%	39	7	
Fixed 8 fields	390	26.8%	40	8	
Fixed 12 fields	531	26.7%	39	27	
Rule-based control	491	27.2%	39	36	
Decision rule	450	27.4%	40	24	
Qwen3-Next 80B (16 of 50 slides counted)	81	18.3%	12	45	19
Qwen3-VL 235B	239	36.6%	36	1	98
Mistral Devstral 2 123B	264	32.4%	36	6	88
MiniMax M2.5	303	29.2%	39	18	64
Qwen3-Coder-Next (46 of 50 slides counted)	303	49.3%	31	23	60
GPT-OSS 120B	390	29.9%	40	15	64
Claude Sonnet 4.6	393	26.8%	39	14	71
Qwen3 32B	479	26.5%	39	29	37
Claude Haiku 4.5	524	25.8%	40	20	58
DeepSeek V3.2	557	25.5%	38	39	20
Nvidia Nemotron Nano 3 30B	568	25.0%	38	50	0
Llama 4 Maverick	575	25.2%	39	46	10
Amazon Nova Pro	587	24.8%	39	50	0
Ministral 3 14B	587	24.8%	39	50	2
Gemma 3 27B	587	24.8%	39	50	0

The agent with Claude Sonnet 4.6 reads **393 fields against 450** for the decision rule, at the same density error. GPT-OSS 120B, with open weights, reads 390. Claude Haiku 4.5 reads 524, the same as the WHO protocol.

A fixed 8 fields does as well on the total. It reads 390 fields at 26.8% error and sends 8 slides to a person, against the agent's 393, 26.8% and 14. On expert counts it is ahead: 7.1% against 8.1%. No model is needed to read a flat eight fields, so a total is not where the agent earns its place.

What separates the models is not their size. It is whether they are willing to stop. Nova Pro, Gemma 3 and Nemotron never chose `stop` in more than a thousand calls each. They read every field the slide has and send the slide to a person when the fields run out, so they cost more than the WHO protocol. Llama 4 Maverick and Ministral 3 stop a handful of times and reach the same place. Three models stop too early. Devstral 2, MiniMax M2.5 and Qwen3-VL read fewer than 310 fields and their density error rises to between 29% and 37%. Two Qwen

models skip fields instead of counting them. Qwen3-Next skipped 663 times and counted on only 16 slides, so its error is not comparable. Qwen3-Coder-Next skipped 220 times and reached 49.3% error.

Where the fields go is the difference. The fixed policy reads eight fields on every slide whether the parasites are everywhere or almost absent: on the ten densest slides it reads 8 and on the ten sparsest 8, and its field count barely tracks the slide (r = 0.12 against expert density). The agent spends the same budget unevenly. It reads 3 fields on the ten densest slides and 9.5 on the ten sparsest, from 1 to 14 across the set, r = -0.70. It stops after one field when the interval is already inside a band and keeps reading when it is not.

The measured claim is this. At the same cost and the same error, the agent puts its effort where the evidence is thin and states a reason for every move. That reason sits on the demo page beside the field it was made on. Against an oracle counter it is behind both the rule (8.1% against 4.7%) and the fixed policy. Stopping earlier costs accuracy on exact counts.

Generalisation, each scored once

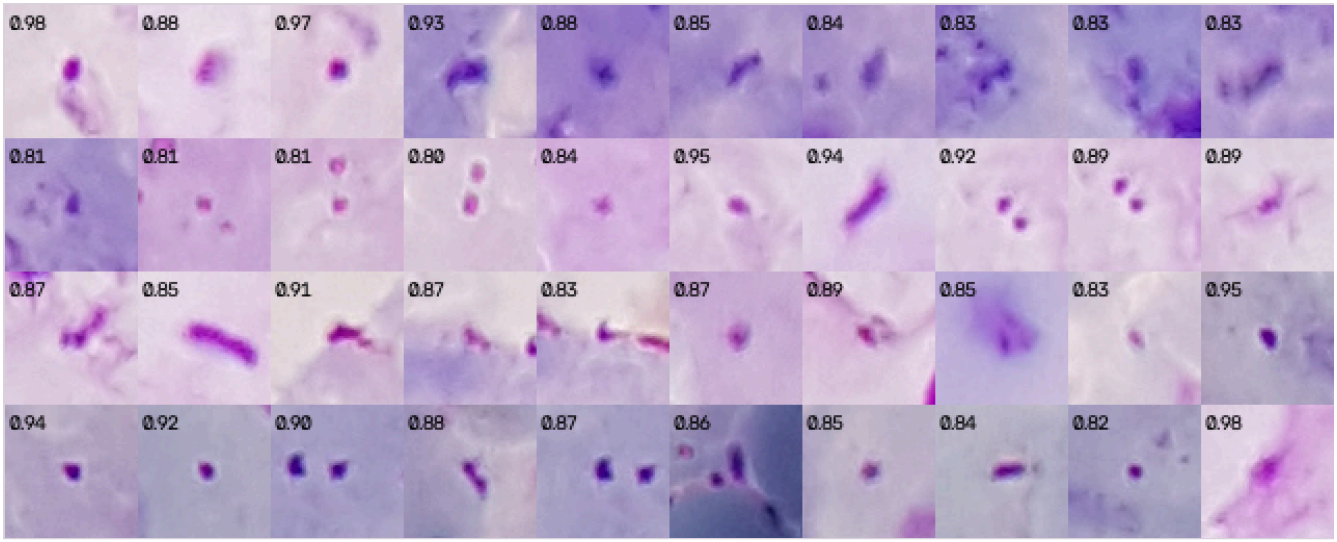
Another laboratory. A CC0 Tanzanian set with a different microscope, camera and stain, scored at commit `b8995e0`. Image-level AUC **0.850** over 1,129 infected and 1,061 uninfected images. 99.3% of uninfected images return zero parasites.

Another species. The falciparum detector on *P. vivax*, 90 patients, 1,804 images. **F1 0.292** against 0.790 on falciparum. 26 patients under 10% recall. The median score on correct detections is 0.685. On incorrect detections it is 0.710. The score does not separate them. No automatic species warning is possible from this detector.

8 Failure modes

Figures in `docs/figures/`. Catalogue in `docs/results-parasites.md`.

1. **Sparse slides.** With few parasites, false positives dominate. This sets the 500/µL limit of detection and the 148/µL noise floor.
2. **Platelets and stain debris.** The commonest false positive is a small round purple object. The patch network removes most. The rest are in `docs/figures/parasite-false-positives-sparse-fields.png`.
3. **Other species.** F1 0.292 on *P. vivax*, with no usable confidence signal.
4. **Soft fields.** On a virtual stage, `refocus` can only choose among captured focus planes. On hardware it is untested.



The worst false positives on sparse fields at the shipped threshold.

9 COOL on Graviton

Four builds measured on real instances. Every result file is in `bench/results/` with the `cv.getBuildInformation()` lines of the binary that ran.

The pip wheel against the Marketplace COOL image, as recorded on the instance.

	PIP WHEEL	COOL IMAGE
OpenCV	`5.0.0-dirty`	`4.13.0-153-g9a6e0d1bff-dirty`
Custom HAL	YES (carotene (ver 0.0.1) KleidiCV (ver 26.03))	YES (carotene (ver 0.0.1) KleidiCV (ver 0.7.0))
Parallel framework	pthreads	TBB (ver 2022.1 interface 12150)
Baseline	NEON FP16	SVE NEON FP16 NEON_DOTPROD NEON_FP16 NEON_BF16

Carotene is in both builds. The pip wheel's KleidiCV is newer than the COOL image's. The COOL image is one major OpenCV version behind.

Function benchmark

Geometric mean of ten per-function speedups against the pip wheel, same machine: COOL **1.59×**. OpenCV 5 with the Arm HAL compiled out: 0.73×. x86 `c7i.8xlarge`: 0.88×.

Whole pipeline

Twelve real NLM fields, the same twelve on every build.

BUILD	SECONDS / FIELD	SLIDES / HOUR
OpenCV 5, Arm HAL compiled out	0.214	567
OpenCV 5 pip wheel	0.222	547
COOL image	0.852	143

The COOL image is 3.8× slower on the pipeline. Three of seven stages are faster on it and three are slower by 7 to 19%. The candidate stage is 9.5× slower. That stage runs a 31×31 ellipse over `float32` at twelve megapixels. The table below times its real calls at that size, at 32 threads and at 1.

Candidate-stage calls, 4032×3024, median of 10 runs, milliseconds.

CALL	WHEEL @32	COOL @32	WHEEL @1	COOL @1	WHEEL GAIN	COOL GAIN
cvtColor BGR2Lab	1.67	1.50	32.38	32.33	19.4×	**21.6×**
GaussianBlur, float32	1.62	25.12	24.11	24.85	14.9×	**1.0×**
morphologyEx TOPHAT, float32, ellipse 31	54.72	1,125.82	1,132.53	1,133.34	20.7×	**1.0×**
dilate, float32, ellipse 15	5.07	130.56	133.80	132.25	26.4×	**1.0×**
morphologyEx TOPHAT, uint8, ellipse 31	20.56	339.52	345.51	343.04	16.8×	**1.0×**

At one thread the two builds agree within 6% on every call. At 32 threads the pip wheel gains 15 to 25× on the filter and morphology calls. The COOL image gains nothing on them. `cvtColor` parallelises equally on both, so TBB is working. The HAL-intercepted filter and morphology paths do not use the threads.

The workaround

The library runs the morphology on one core, so the pipeline hands it work one core can do. `parasites.py` splits the image into horizontal bands, one per thread, each band carrying a halo as wide as the kernel radius times the number of chained passes, and runs the top-hat and the dilation band by band. Thirty pixels of halo for the top-hat, seven for the dilation. The result is checked against the unsplit call before it is timed, so a wrong answer cannot be reported as a fast one. It is forty lines behind `AUTOSCOPE_TILED_MORPH=1`, off by default.

With it on, the candidate stage falls from 0.718 s to 0.114 s and the COOL image reads 532 slides an hour against the pip wheel's 547.

The same answers, on both builds

A speed number from a different library only describes AutoScope if the library gives back the same answer. The whole pipeline was run twice on the same 12 fields, once on the pip wheel and once on the COOL image with the workaround on, keeping the parasite and white-cell count of every field.

The same twelve fields, both builds, counts kept.

	PIP WHEEL ON `C8G`	COOL IMAGE ON `C8G`
OpenCV	`5.0.0`	`4.14.0-pre`
Seconds per field	0.218	0.225
Slides per hour	559	540
Fields counted differently	—	0 of 12

605 parasites and every white cell, field by field, identical on both builds. The COOL image runs AutoScope at 0.97× the pip wheel's speed and returns the same counts.

The wheel reads 559 slides an hour in this pair and 547 in the table above. Same build, same twelve fields, two runs on two Spot instances a few days apart: about 2% between them, which is the run-to-run spread on this measurement. Both result files are in `bench/results/`. Comparisons are made inside a pair, never across two runs.

What a slide costs

Prices from the AWS Pricing API on 2026-10-10, in `bench/pricing.json`. One Batch job gets 4 vCPU, so a 32-vCPU instance runs eight slides at once; the per-field time is scaled to a 4-vCPU slot before the division.

Dollars per 1,000 slides, `c8g.8xlarge` in `us-east-1`.

BUILD	ON DEMAND	SPOT
OpenCV 5, Arm HAL compiled out	$2.25	$0.90
OpenCV 5 pip wheel, what production runs	$2.33	$0.93
COOL image with the workaround	$2.40	$0.96

A thousand slides cost about a dollar on Spot, and the three builds are within seven cents of each other. The build with the Arm HAL compiled out is the cheapest of the three, by the same margin by which it is the fastest on this pipeline. Cost is not where the choice between these builds gets made.

What production runs, and why

The Batch container runs the OpenCV 5 pip wheel on `c8g`, Graviton4. That wheel is built with carotene and KleidiCV 26.03, which is the same Arm acceleration the COOL image carries and one version newer. So the production path is Arm-accelerated OpenCV on Graviton; the COOL Marketplace image is the fourth measured arm, and with the workaround it is interchangeable with the wheel, at 0.97× and the same counts.

The defect is in the shipped image, not in the idea. It is reported upstream with the reproduction, opencv/opencv#30373. An OpenCV maintainer triaged it on 21 October, labelled it `category: imgproc` and assigned it to two of the maintainers who work on the HAL. The body of that report is generated from `bench/results/` by `bench/upstream_issue.py`, so it cannot drift from the files it tells a reader to check, and `bench/repro_hal_threads.py` reproduces the behaviour in one file with no repository, no AWS and no Marketplace subscription.

```
GROUP=pipeline THREADS=0,1 COOL_AMI=<ami> bash bench/run_on_ec2.sh c8g.8xlarge c8g-cool 10
  GROUP=pipeline THREADS=0,1                bash bench/run_on_ec2.sh c8g.8xlarge c8g-stock 10
  PROFILE=1                                 bash bench/run_on_ec2.sh c8g.8xlarge c8g-stock-counts
  PROFILE=1 TILED_MORPH=1 COOL_AMI=<ami>     bash bench/run_on_ec2.sh c8g.8xlarge c8g-cool-counts
  python bench/compare_counts.py profile-c8g-stock-counts profile-c8g-cool-counts
```

10 Limits and responsible use

Screening aid

A person reviews every output. The trace, the crops and the interval are shown so the technician can disagree. AutoScope has not been clinically validated and has not been run on a clinical sample. It must not be used to make a treatment decision.

Virtual stage

No microscope or motorised stage was available. `refocus` and the stage moves select among fields already captured. The comparison between policies holds, because every policy sees the same fields. Stage control on hardware is untested.

Data

Development and most evaluation use NLM/LHNCBC thick films from one collection. The Tanzanian set is the only transfer evidence. AUC 0.850 there is well below the within-collection figures. A deployment would need local calibration slides. The threshold-choosing script is in the repository for that purpose.

Species

Falciparum only. On *P. vivax* the detector reads F1 0.292 and its confidence score does not indicate failure. It cannot be used where vivax circulates.

Test-set discipline

The 50 held-out patients were fixed before any parasite work and never changed. The uninfected frozen half, the Tanzanian set and the vivax set were each scored once. Re-scoring requires an explicit flag and a statement in this report. The threshold was chosen on 20 calibration patients by a stated rule.

The guards, and where each one is

Every limit is outside the model. A policy that keeps answering "one more field" stops at the state machine, not when it decides to.

Operational limits. `docs/operations.md` has the rest.

GUARD	VALUE	WHERE
Fields on one slide	400	`agent.tf`, the `Limits` choice
Time on one slide	1 h	`agent.tf`, `TimeoutSeconds`
One field read	15 min, 3 tries	`batch.tf`
One agent call	2 min	`agent.tf`, and anything it raises goes to a person
Spot fleet	64 vCPU, 0 at rest	`batch.tf`
API	10/s, burst 20	`api.tf`
Sessions started	alarm over 50 / 5 min	`budget.tf`
Spend	$100/month, $25 Bedrock	`budget.tf`, email at 80% forecast
Uploads	deleted after 30 days	`storage.tf`

The agent has no move that issues a diagnosis, and no path to one. Its five moves are count, refocus, skip, stop and `flag_for_human`; the call comes from `decide()` reading the counts. If the agent Lambda raises anything at all, the slide goes to a person with the fields read so far already in the trace. The reading no longer fails with no outcome at all.

Privacy

All imagery is public, retrospective and de-identified. No patient identifier is processed, stored or transmitted. Fields uploaded to the demo go to a private bucket and are not published.

11 What would make the detector stronger

Measured on the cached candidate scores. Nothing here is shipped: each item changes counts, which would invalidate every recorded model call and every frozen figure above.

What limits it

The density error tracks the per-slide spread of the found-to-true parasite ratio, not F1. The spread is false alarms, not recall: across the 50 held-out slides the false-alarm rate per true parasite varies with a coefficient of variation of 86%, recall with 20.1%. Nine slides carry 73.7% of that variance. The other 41 read 11.6% error with 15.4% spread. Six of the nine are sparse slides that over-count on detections scored just above the threshold: rods, streaks and clumps. Three are dense slides with soft focus whose parasites score near zero. Every counter trained for this project fails on the same nine.

Count by score, not by one each

A false alarm that clears the threshold usually clears it by little. Summing the scores of the kept detections instead of counting them, and rescaling by one factor fitted on the 20 calibration patients (k = 1.337), gives:

Held-out 50 patients. k is the only fitted quantity and it was fitted on calibration.

	ERROR	BIAS	PER-SLIDE SPREAD	WITHIN ±25%
shipped count	19.3%	-4.1%	28.8%	70%
soft count	**17.7%**	-1.3%	**25.1%**	**76%**

The soft count also removes the cancellation described in Section 7: with an exact white cell count the shipped count gets worse and the soft count does not. It costs a few dozen lines and no training. It is the first thing to build.

Context around the patch

The parasites the judge rejects are fainter than the false alarms it accepts. That is the judge's own axis. Features of the surroundings, neighbours, white cells, field debris, do not separate the two error sets. A wider patch or a second-stage classifier on context is not worth building.

A single-stage detector

The requirement is per-slide spread under 20% at bias under 5% on the 20 calibration patients, with the sparse and the dense failure modes both inside ±25%. F1 does not predict this. The box detectors trained on Colab need their validation split separated from the calibration patients before their numbers can be compared, then one GPU run scored on density.

12 Reproduction

```
make setup                # venv, pinned requirements, opencv-python 5.0.0.93
  make test                 # unit tests
  make data                 # NLM thick smear set, 3.9 GB

  SKIP_TRAIN=1 bash eval/chain.sh          # every held-out and specificity number
  .venv/bin/python eval/agent_eval.py --policies all   # the whole agent table,
                                           # 18 arms replayed from the cache, free
  .venv/bin/python eval/agent_moves.py     # the move each model chose, counted
  bash bench/run_on_ec2.sh c8g.8xlarge c8g-stock   # one benchmark arm
  .venv/bin/python bench/cost_model.py --markdown  # dollars per 1,000 slides
  .venv/bin/python docs/report/build_report.py     # this document
```

The agent comparison needs no AWS account. The benchmark arms need one. Each launches one instance, measures it and terminates it.

CLAIM	EVIDENCE
Detector and failure cases	`docs/results-parasites.md` , `docs/figures/`
Specificity and decision rule	`docs/results-negatives.md`
Agent against the rules	`docs/results-agent.md` , `eval/results/agent_eval.json`
Other laboratory, other species	`docs/results-tanzania.md` , `docs/results-vivax.md`
COOL, four builds	`docs/results-cool.md` , `bench/results/`
What would make the detector stronger	`eval/soft_count.py` , `docs/detector-levers-2026-10-16.md`
Deployed stack	`infra/terraform/` , `infra/terraform/deployed.json` , `docs/aws-setup.md`
Running it, and turning it off	`docs/operations.md`
COOL gives the same counts	`bench/compare_counts.py` , `bench/results/profile-c8g-*-counts.json`

AutoScope · Uriel Nguefack Yefou and Alex Bruno Kenfack Anafack · OpenCV AI Competition 2026, powered by AWS · report built 24 October 2026.

A screening aid, not a diagnostic device. Not clinically validated. Every result is reviewed by a person. Malaria burden figures are from the WHO *World malaria report 2025*, data year 2024. All imagery is public, retrospective and de-identified.